Implementación y análisis de algoritmos de ordenamiento en el microcontrolador ATmega328P

Sánchez Valdéz Alejandra, Urrutia Torres Jenifer Alessandra
Universidad Nacional Autónoma de México
Facultad de Estudios Superiores Cuautitlán

***Resumen*—En el presente trabajo se implementa un programa en lenguaje ensamblador para el microcontrolador ATmega328P con el objetivo de generar un conjunto de números pseudoaleatorios y aplicar algoritmos de ordenamiento para analizar su desempeño. Se generaron 100 valores que fueron almacenados en memoria y posteriormente ordenados mediante los algoritmos Bubble Sort y Selection Sort. El desarrollo del programa se realizó utilizando el entorno de simulación Microchip Studio, permitiendo observar el comportamiento del sistema y verificar el correcto almacenamiento y ordenamiento de los datos en la memoria RAM. Asimismo, se realizaron mediciones del tiempo de ejecución de cada algoritmo mediante el análisis del número de instrucciones ejecutadas, con el propósito de comparar su rendimiento. Finalmente, se presentan los resultados obtenidos mediante tablas y gráficas que permiten analizar la eficiencia de ambos algoritmos.**

I. Introducción

El ordenamiento de datos es una de las operaciones fundamentales dentro del área de la informática y la programación, ya que permite organizar conjuntos de información para facilitar su procesamiento, búsqueda y análisis. Existen diversos algoritmos de ordenamiento que presentan diferentes niveles de eficiencia dependiendo del tamaño del conjunto de datos y de la forma en que se implementan. Por esta razón, el estudio y comparación de estos algoritmos resulta de gran importancia en el desarrollo de sistemas computacionales.

En el contexto de los sistemas embebidos, la implementación de algoritmos adquiere un interés adicional debido a las limitaciones de recursos como memoria y capacidad de procesamiento. Los microcontroladores son dispositivos ampliamente utilizados para el control y procesamiento de información en múltiples aplicaciones electrónicas, por lo que analizar el comportamiento de algoritmos dentro de este tipo de arquitecturas permite comprender mejor su rendimiento y optimización.

En el presente trabajo se realiza la implementación de dos algoritmos de ordenamiento, Bubble Sort y Selection Sort, utilizando lenguaje ensamblador en el microcontrolador ATmega328P. Inicialmente se generan números pseudoaleatorios que son almacenados en memoria para posteriormente ser ordenados mediante ambos algoritmos. A través del uso del entorno de simulación Microchip Studio se analizan los resultados obtenidos y se mide el número de instrucciones ejecutadas por cada algoritmo, con el fin de comparar su desempeño y eficiencia. Finalmente, los resultados se presentan mediante tablas y gráficas que permiten visualizar las diferencias entre ambos métodos de ordenamiento.

II. Algoritmos de ordenamiento

Los algoritmos de ordenamiento son procedimientos utilizados en Ciencias de la Computación para organizar un conjunto de datos en un orden específico, normalmente ascendente (de menor a mayor) o descendente (de mayor a menor).

Un algoritmo de ordenamiento toma una lista de elementos (por ejemplo, números) y reorganiza su posición siguiendo una regla de comparación.

Para hacerlo, el algoritmo compara valores entre sí y los intercambia hasta que toda la lista queda ordenada.

Ejemplo:
Lista original:
8, 3, 5, 1, 9
Lista ordenada:
1, 3, 5, 8, 9

Cada algoritmo utiliza una estrategia diferente para realizar comparaciones e intercambios entre los elementos.

III. Importancia de los algoritmos de ordenamiento

Los algoritmos de ordenamiento son importantes porque:

- Facilitan la búsqueda de información.
- Mejoran el rendimiento de los programas.
- Se utilizan en múltiples aplicaciones.

Entre sus aplicaciones se encuentran:

- Bases de datos
- Sistemas de archivos
- Procesamiento de datos
- Inteligencia artificial

Además permiten analizar y optimizar algoritmos, ya que son un caso clásico para estudiar conceptos como complejidad algorítmica y eficiencia computacional.

IV. Notación Big-O

La notación Big-O se utiliza en Ciencias de la Computación para describir qué tan eficiente es un algoritmo cuando el tamaño de los datos de entrada aumenta.

Permite medir cómo crece el tiempo de ejecución o el uso de memoria de un algoritmo conforme aumenta la cantidad de datos que procesa.

Por ejemplo:

- $O(n)$: el tiempo crece proporcionalmente al número de datos.

- $O(n^2)$: el tiempo crece mucho más rápido cuando aumentan los datos.
- $O(n \log n)$: es más eficiente que $O(n^2)$ para listas grandes.

Algoritmo de ordenamiento	Mejor Caso	Caso Promedio	Peor Caso	Complejidad Espacial
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(n log n)
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)

V. Tiempo del algoritmo 1

Para el primer algoritmo Bubble Sort en total se obtuvieron:

1,420,940 ciclos

$$Tiempo = \frac{Ciclos}{Frecuencia}$$

$$Tiempo = \frac{1{,}420{,}940\ ciclos}{16MHz}$$

$$Tiempo = 88.80ms$$

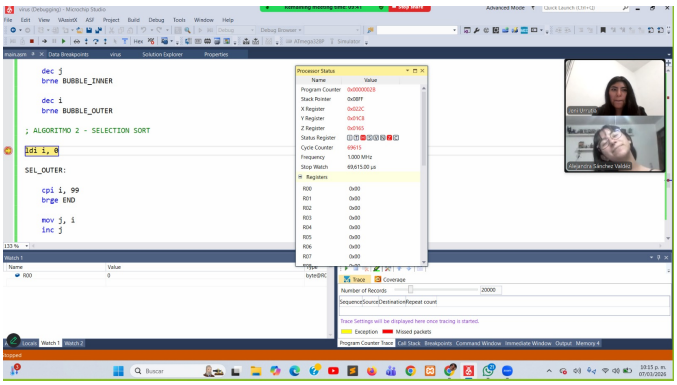

Figura 1. Tiempo de ejecución del algoritmo Bubble Sort cuando inicia

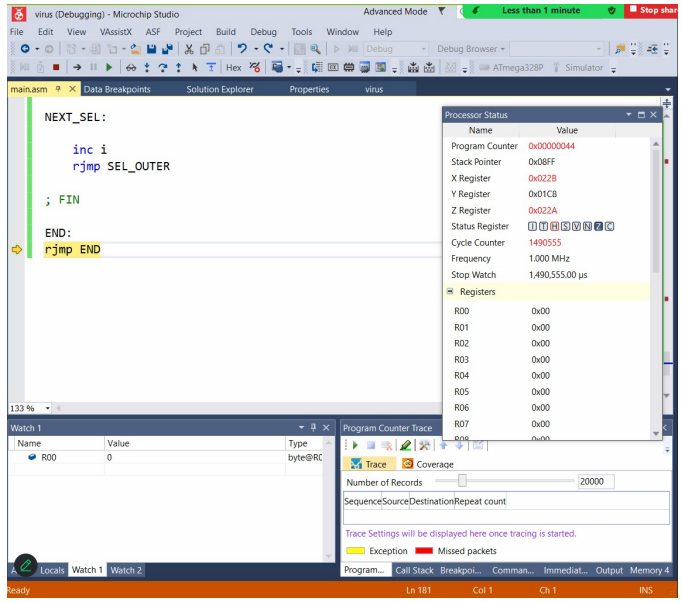

Figura 2. Tiempo de ejecución del algoritmo Bubble Sort cuando termina

VI. Tiempo del algoritmo 2

Para el segundo algoritmo Selection Sort se obtuvieron:

7,573,416 ciclos

$$Tiempo = \frac{Ciclos}{Frecuencia}$$

$$Tiempo = \frac{7{,}573{,}416\ ciclos}{16MHz}$$

$$Tiempo = 473.33ms$$

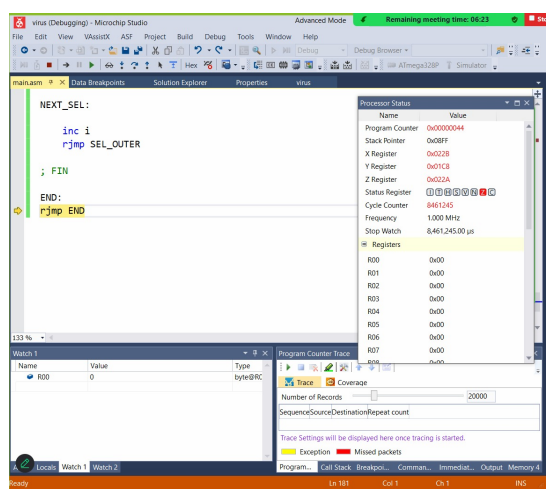

Figura 3. Tiempo de ejecución del algoritmo Selection Sort cuando inicia

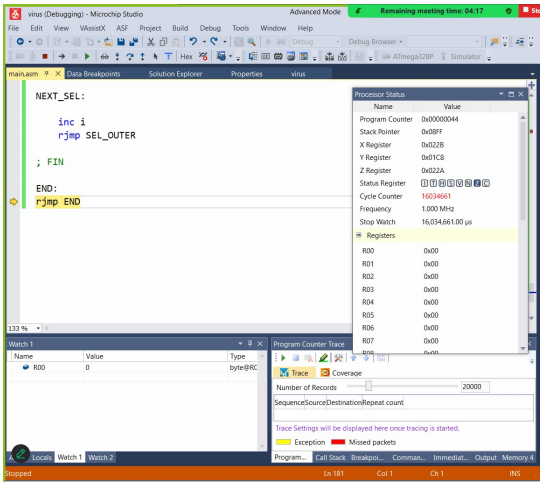

Figura 4. Tiempo de ejecución del algoritmo Selection Sort cuando termina

VII. Comparación de los resultados obtenidos con los teóricos

Desde el punto de vista teórico, ambos algoritmos presentan una complejidad temporal:

$$O(n^2)$$

Esto significa que el número de operaciones crece proporcionalmente al cuadrado de la cantidad de elementos a ordenar.

Los resultados obtenidos son consistentes con el análisis teórico, ya que ambos algoritmos presentan un comportamiento cuadrático. Sin embargo, en esta implementación específica Bubble Sort resultó más eficiente que Selection Sort.

VIII. Gráficas con los resultados

Algoritmo	Número de datos	Ciclos ejecutados	Tiempo(ms)
Bubble Sort	100	1,420,940	88.8
Selection Sort	100	7,573,416	473.33

Figura 5. Comparación de ciclos

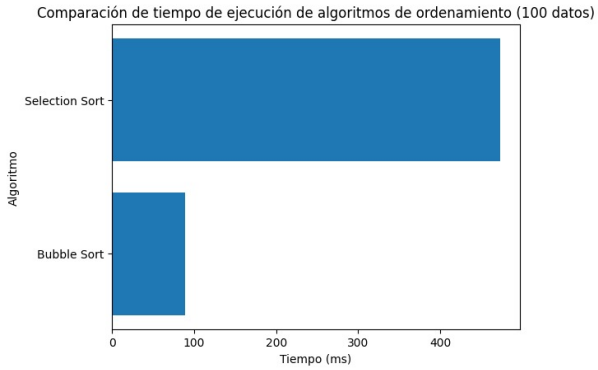

Figura 6. Comparación de tiempo de ejecución

IX. Pseudocódigo

IX-A. *Generación de números pseudoaleatorios*

```
Inicio

Definir arreglo table_of_unsorted_numbers de tamaño 100
Definir variable rand
Definir contador i

rand ← 37
i ← 100

Mientras i > 0 hacer

    rand ← rand << 1        // desplazamiento a la izquierda
    rand ← rand XOR i       // operación XOR para generar variación

    table_of_unsorted_numbers[100 - i] ← rand

    i ← i - 1

Fin Mientras
```

Figura 7. Pseudocódigo generación de números

IX-B. *Pseudocódigo Bubble Sort*

```
Inicio

Para i desde 99 hasta 1 hacer

    Para j desde 0 hasta i-1 hacer

        Si arreglo[j] > arreglo[j+1] entonces

            intercambiar arreglo[j] y arreglo[j+1]

        Fin Si

    Fin Para

Fin Para

Fin
```

Figura 8. Pseudocódigo Bubble Sort

IX-C. *Pseudocódigo Selection Sort*

```
Para i desde 0 hasta 98 hacer

    min ← i

    Para j desde i+1 hasta 99 hacer

        Si arreglo[j] < arreglo[min] entonces

            min ← j

        Fin Si

    Fin Para

    intercambiar arreglo[i] y arreglo[min]

Fin Para

Fin
```

Figura 9. Pseudocódigo Selection Sort

X. Análisis y comentarios de resultados

Durante la ejecución del programa en el simulador Microchip Studio se generaron 100 números pseudoaleatorios que fueron almacenados en la memoria RAM del microcontrolador ATmega328P. Posteriormente se aplicaron dos algoritmos de ordenamiento: Bubble Sort y Selection Sort con el objetivo de comparar su desempeño.

Al observar las tablas en la memoria del simulador se comprobó que los números generados inicialmente se encontraban desordenados, mientras que las tablas resultantes mostraban los valores ordenados de forma ascendente.

Teóricamente ambos algoritmos presentan una complejidad temporal de $O(n^2)$, lo que significa que el número de operaciones crece de forma cuadrática conforme aumenta la cantidad de datos.

Referencias

[1] freeCodeCamp.org, "Sorting Algorithms Explained with Examples in Python, Java, and C++." [Online]. Available: https://www.freecodecamp.org/news/sorting-algorithms-explained-with-examples-in-python-java-and-c/

[2] freeCodeCamp.org, "Big O Notation Explained with Examples." [Online]. Available: https://www.freecodecamp.org/news/big-o-notation-explained-with-examples/

[3] A. Bhuyan, "Understanding the Role of an Assembler in Computer Architecture," DEV Community.